

BD NoSQL Orientée Graphe

« Neo4j »

Pr. SOUSSI Nassima

Introduction sur la BD Neo4j



Présentation générale

- **Neo4j** est une base de données orientée graphe, libre et écrite en Java.
- Développée par Neo Technology (une société suédoise): les premières lignes de codes datent de l'année 2000 et la version 1.0 est sortie en 2010.
- Elle est l'une des premières bases de données orientées graphes, mais aussi l'une **des plus évoluées et robustes**.



Caractéristiques

- **Haute disponibilité** : Fonctionnement en cluster possible (peut répartir la charge sur plusieurs serveur)
- **Volumétrie** : stocker et requêter des milliards de nœuds et de relations;
- **Cypher** : langage d'interrogation de la base à travers HTTP => Langage très simple et très puissant
- **Schemaless** : pas de schéma préétabli.

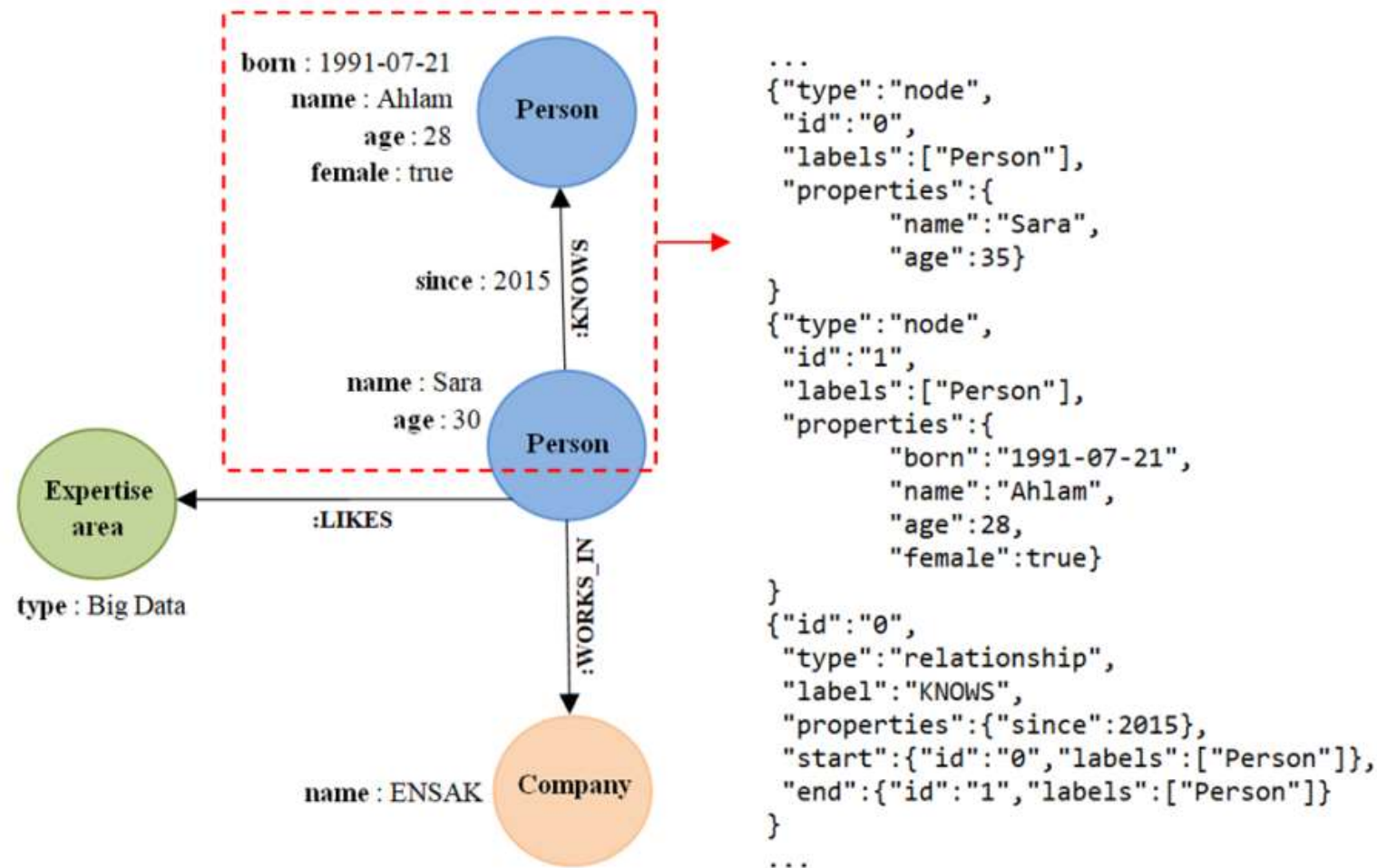


Modèle de données : **Eléments de base**

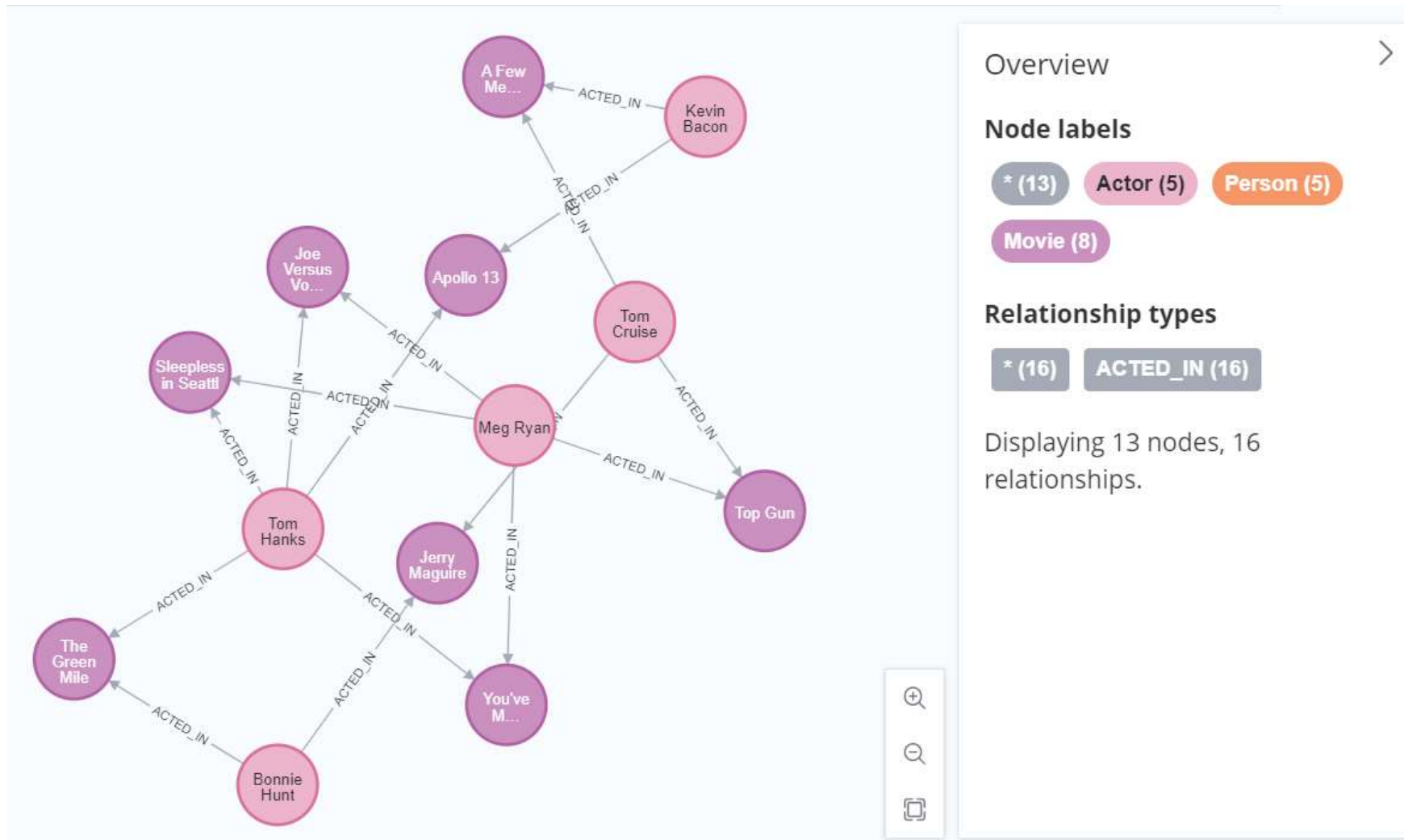
- Le **nœud** est l'unité fondamentale qui constitue un graphe :
Peut être associé à des **labels** et peut posséder des **propriétés**
Un nœud peut posséder plusieurs labels
- Une **relation** connecte deux nœuds avec une direction : Peut également posséder des propriétés.
- **Propriétés** pour les **nœuds** et **relations** comme paires clé-valeur : Valeurs de type nombre, chaîne de caractères, booléen et listes.



Modèle de données : Exemple 1



Modèle de données : Exemple 2



Cypher Query Language



Le langage **Cypher**

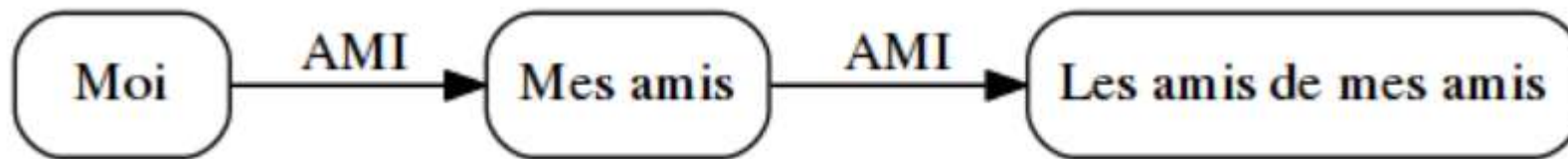
- Cypher est un langage déclaratif permettant de requêter et mettre à jour le graphe.
- Inspiré du SQL, on y retrouve beaucoup de concepts familiers, comme les clauses *WHERE*, *ORDER BY*, *SKIP*, *LIMIT*...
- Son objectif est de permettre à l'utilisateur de définir des **motifs**, qui seront par la suite recherchés dans tout le graphe.



Le langage Cypher

Exemple : je veux les amis de mes amis.

⇒ Dans ce cas il faut décrire le pattern suivant :



⇒ le pattern textuelle est comme suit :

```
(moi) -[:AMI]-> (mesAmis) -[:AMI]-> (amisDeMesAmis)
```

Le langage Cypher : **Noeud**

- Les **nœuds** sont représentés avec des **parenthèses**, ce qui ressemble à des cercles : **()**
- Pour spécifier un label, il suffit de l'ajouter comme ceci : **(monNoeud:monLabel)**
- Si vous avez besoin d'identifier le nœud dans votre requête (dans une clause WHERE par exemple), il suffit de lui donner un nom : **(monNoeud)**



Le langage Cypher : **Noeud**

Voici quelques exemples :

- **()** : n'importe quel nœud ;
- **(:Personne)** : un nœud avec le label *Personne*;
- **(n:Personne)** : un nœud identifié dans la variable *n* avec le label *Personne* ;
- **(n:Personne:Acteur)** : un nœud identifié dans la variable *n* avec le label *Personne* et *Acteur*.



Le langage Cypher : **Relation**

- Les **relations** sont représentées par deux tirets avec un '>', ce qui ressemble à une flèche : **-->**
- Si vous avez besoin d'identifier la relation dans votre requête, vous pouvez lui donner un nom comme ceci : **-[maRelation]->**
- Pour spécifier le type de la relation, il suffit de l'ajouter comme ceci : **-[maRelation:MON_TYPE]->**



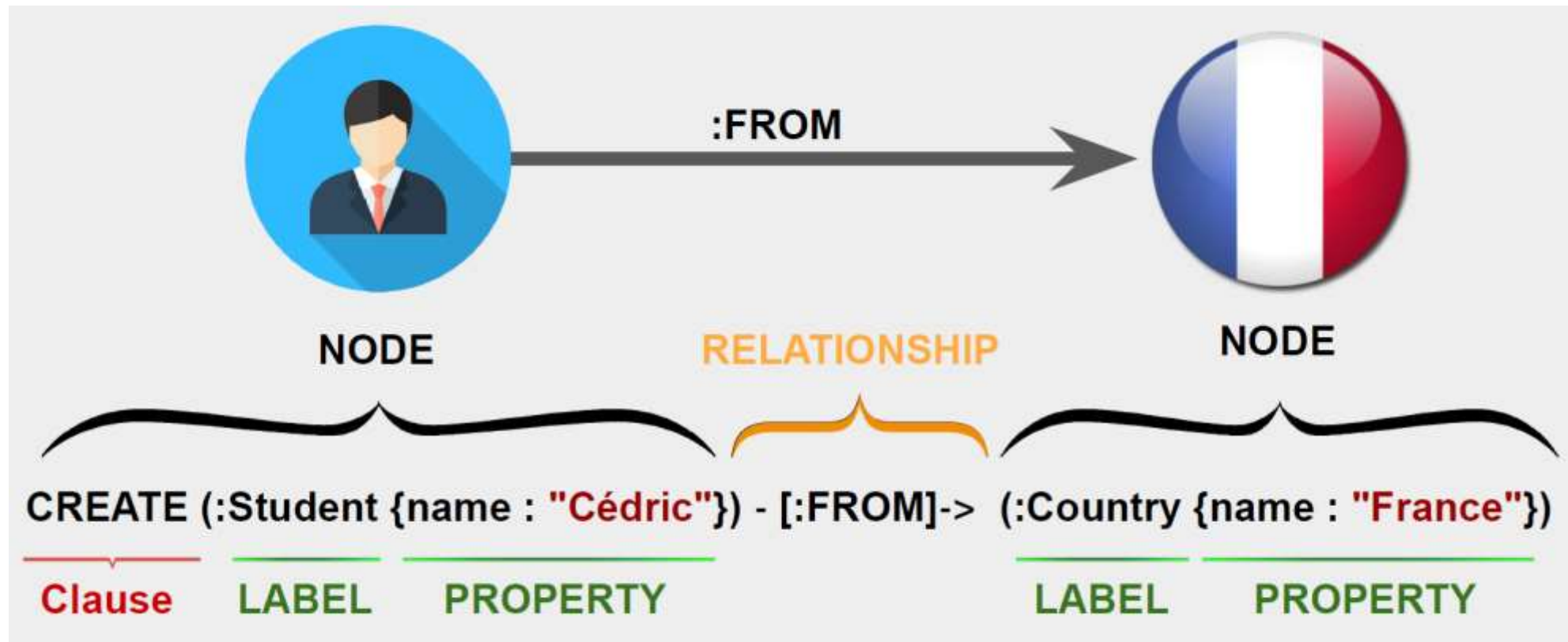
Le langage Cypher : Relation

Voici quelques exemples :

- **(a)--(b)** : n'importe quelle relation entre le nœud *a* et *b* (peu importe la direction) ;
- **(a)-[:AMI]->(b)** : relation de type *AMI* depuis le nœud *a* vers le nœud *b* ;
- **(a)-[r:AMI|CONNAIT]->(b)** : relation identifiée dans la variable *r* de type *AMI* ou *CONNAIT* depuis le nœud *a* vers le nœud *b*.



Le langage Cypher : Exemple de Pattern



Le langage Cypher : Création

- **CREATE** : crée des nœuds, des relations et des patterns
- **MERGE** : crée aussi des nœuds, des relations et des modèles si le pattern n'existe pas.

Le langage Cypher : Création

■ Création d'un nœud :

1. **CREATE** (a:student{ name: 'Mohamed Amine', DateNais : 1999})
2. **CREATE** (b:student{name: 'Sara Adam', DateNais : 2001}),
(t:teacher:director{ name: 'Ahmed Fizazi', age:49})
3. **MERGE** (a:student{ name: 'Mohamed Amine', DateNais : 1999})

Le langage Cypher : Création

- **Création d'une relation :**

MATCH (a:student), (t:teacher)

WHERE t.name = 'Ahmed Fizazi' **and** a.name='Mohamed Amine'

CREATE (t)-[r:enseigne]->(a)

RETURN a,r,t

Le langage Cypher : **Lecture**

- **MATCH** : le moyen le plus simple d'obtenir des données à partir du graphe

```
MATCH (s:Student)
RETURN s.name AS Name, s.hair AS Hair
```

- **WHERE** : Filtrer les résultats à l'aide de conditions

```
MATCH (s:Student)
WHERE s.name = "Andrei"
RETURN s.hair AS Hair
```

=

```
MATCH (s:Student {name : "Andrei"})
RETURN s.hair AS Hair
```



Le langage Cypher : **Lecture**

- **RETURN :**
 - Permet de retourner le résultat voulu.
 - Peut être utilisé avec **SKIP/LIMIT** et **ORDER BY**

```
MATCH (s:Student)
RETURN s.name, s.birthDate
ORDER BY s.birthDate DESC
LIMIT 10
```



Le langage Cypher : **Lecture**

■ **RETURN :**

- Les données retournées peuvent être des : nœuds, propriétés ou patterns.

```
MATCH p = (:Student)-[:FROM]->(:Country)  
RETURN p
```

- Affichage de tout le graphe :

```
MATCH (n) RETURN (n)
```



Le langage Cypher : Ajouter

- SET :

- Ajoute ou met à jour les **propriétés** (*sur les nœuds et les relations*) et les **labels** (*uniquement sur les nœuds*)

```
MATCH (s:Student {name : "Andrei"})
```

```
SET s:Person
```

```
SET s.birthYear = 1990
```



Le langage Cypher : **Suppression**

■ **DELETE :**

- Supprimer des nœuds et des relations.
- Lors de la suppression d'un nœud, utilisez **DETACH** pour supprimer d'abord ses relations.

```
MATCH (s:Student {name : "Andrei"})  
DETACH DELETE s
```

- Supprimer tous le graphe :

```
MATCH (n) DETACH DELETE (n)
```



Le langage Cypher : **Suppression**

- **REMOVE :**

Supprimer les **propriétés** d'un nœud ou d'une relation.

```
MATCH (s:Student {name : "Andrei"})  
REMOVE s.birthYear
```



Le langage Cypher

Documentation officielle :

<https://neo4j.com/docs/cypher-manual/current/clauses/>

